

Under the Hood of Windows 8

>>Windows 8 is highly ambitious in its scope, and comes with a lot of changes under the hood. Should you then, as a developer, be interested?> by Kshitij Sobti

Windows 8 brings the biggest change to the Windows ecosystem yet, and sets the stage for the future of the Windows operating system. Unlike previously, these changes aren't just on the level of a few new APIs here and there. The very way applications are developed for Windows has now changed, and will likely never be the same again. Some of the biggest changes to come with Windows 8 would have to be:

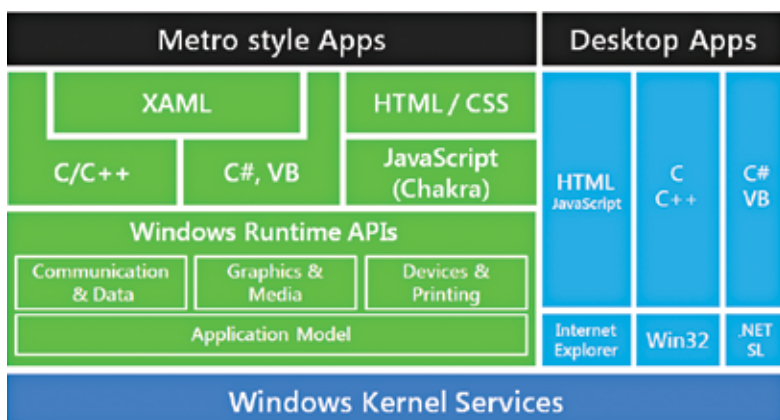
- An entirely new application runtime (WinRT)
- A new interface
- A new Microsoft app store
- Support for ARM-based tablet devices

We will take a look at each of these in turn, starting, of course, with the most divisive of them all, the new UI.

The Modern UI

This is undoubtedly the most controversial addition to Window 8, and one of the major reasons that the Windows ecosystem will never be the same again. Originally called Metro, it has now been renamed to Modern UI (or Windows 8-style UI) due to a legal battle. With this approach to its interface, Microsoft now supports a whole new range of applications under Windows.

These applications are designed to run in one of three states, full screen, snapped, or fill. The first state is obvious; in the second case, the application is snapped to the left or the right of the screen at a width of 320 pixels, and in the last state is of an application filling the remaining area after an application is snapped next to it. Additionally applications can be running in portrait or landscape view.



The many layers of Windows 8

Applications can also feed data to the users even when they are inactive via live tiles. Essentially, an application can be programmed to give some information about its state even when it is not running. This information can be presented on the applications "icon" on the start screen. So an email application can show the unread message count on the start screen and can even give pop-up notifications in case there is a new message. For this purpose, a small part of the application will always be running.

The new style for applications is expected to follow Microsoft's UI guidelines, which put a focus on content over chrome. What this means is that applications should avoid skeuomorphic design patterns such as bevelled buttons and frames and instead use the content itself to distinguish different parts of the UI. For example, instead of unnecessary borders and bevels, the placement and typography of content should be used to express its purpose.

This is the new direction of Windows, and though it is unlikely that traditional applications with menus and title bars is going to go away soon, Microsoft seems to have a lot of confidence in this new vision. To support it, they have an entirely new API for Windows applications, which we come to next.

Windows runtime

WinRT, or Windows Runtime is an API developed for, and available to the new style of applications discussed above. These new APIs allow a much simplified path for those developing Windows applications. However, they are also mostly only available to the new style of applications.

RT-based applications can be written in the usual C#, Visual Basic, and C++, but now there is also the option of web technologies such as HTML, CSS, and JavaScript. Microsoft also provides the Windows Library for JavaScript, which provide JavaScript API for building applications.

However, this new API does not provide all the features that Win32 did before it, and as a result certain kinds of applications are just not possible with the new API. All such applications actually run in a sandbox and have very limited system access. In fact, because browsers need more system-level access than is available to Windows 8 style applications, browser on Windows 8 are given a special exception from the sandbox and can mix normal Windows API and WinRT APIs.